API SIX灰度发布方案

1.方案

1. 前端提供界面通过添加 Cookie,如果设置了https secure 那就后端写个接口添加
2. APISIX 通过 traffic-split 分流流量,将带有 grey_release=true 的 cookie 的请求分流到预发布组中
3. Cookie设置页面隐藏到前端界面中,用户不可能启用,所以永远访问不到测试环境

1.1 需要考虑的问题

1. 测试数据删除问题: percona推荐使用python 脚本分析[https://github.com/danfengcao/binlog2sql]
2. Cookie 读取:使用 traffic-split has 运算符匹配
3. etcd 集群备份恢复
4. apisix 高可用

2.开始使用

2.1 ETCD 集群

> 整理后参照当前docker-compose搭建,建议直接设置环境变量而不是 etcd.yml,所有配置都能找到,启动后通过以下命令检查集群状态

```
# 检查集群状态
etcdctl --write-out=table --endpoints=etcd-1:2379,etcd-2:2379,etcd-3:2379 endpoint status
```

如果正常你将会看到如下内容

ENDPOINT	ID	VERSION	DB SIZE	IS LEADER	IS LEARNER	RAFT TERM	RAFT INDEX	RAFT APPLIED INDEX	ERRORS
etcd-1:2379	4dd145349f1616cf	3.5.2	96 MB	false	false	5	128639	128639	
etcd-2:2379	3124e52d41479d1a	3.5.2	96 MB	false	false	5	128640	128640	
etcd-3:2379	534010c557032af0	3.5.2	96 MB	true	false	5	128641	128641	

2.1.1 启用密码登录

```
# 设置集群用户名/密码 使用强密码 MDVjOTdiMDg0YjQyOGIyNmJiNmE2NzAwNWFhZjY4NzUK
etcdctl  user add root --interactive=false MDVjOTdiMDg0YjQyOGIyNmJiNmE2NzAwNWFhZjY4NzUK
# 分配 root 角色
etcdctl user grant-role root root
# 启动登陆认证
etcdctl auth enable
# 尝试执行 put
etcdctl --endpoints=etcd-1:2379,etcd-2:2379,etcd-3:2379  --user=user:password put foo bar
# 尝试获取
etcdctl --endpoints=etcd-1:2379,etcd-2:2379,etcd-3:2379  --user=user:password get foo
# 尝试部署成功后检查apisix 是否能正常使用
etcdctl --endpoints=etcd-1:2379,etcd-2:2379,etcd-3:2379  --user=user:password get /apisix --prefix --keys-only
```

2.1.2 ETCD 备份恢复

- 缓存快照

```
etcdctl --user=user:password --enndpoint=etcd-1:2379,etcd-2:2379,etcd-3:2379 snapshot save path/to/my.db
```

- 根据快照恢复

> 当通过主动save 获得的快照时,db文件支持数据校验,若通过 etcd 数据目录获得的db 文件需要添加参数 `--skip-hash-check` 跳过校验；另外,要避免db文件修改了集群其他信息

```
etcdctl --user=user:password --skip-hash-check --enndpoint=etcd-1:2379,etcd-2:2379,etcd-3:2379 snapshot restore path/t
```

2.1.3 ETCD 集群测试,基准测试

1. 编译 benchmark

```
# 下载对应版本的etcd
git clone https://github.com/etcd-io/etcd.git
# 使用go mod 下载依赖
go mod download
# 编译通过得到benchmark 工具
cd tools/benchmark; go install
# 将编译好的文件移动到系统PATH目录
mv benchmark /usr/local/bin
# 设置执行权限
chmod +x /usr/local/bin/benchmark
```

2. 使用 benchmark 测试基础性能

> 参考官方基准测试文档

```
# write to leader
benchmark --endpoints=${HOST_1} --target-leader --conns=1 --clients=1 \
    put --key-size=8 --sequential-keys --total=10000 --val-size=256
benchmark --endpoints=${HOST_1} --target-leader  --conns=100 --clients=1000 \
    put --key-size=8 --sequential-keys --total=100000 --val-size=256

# write to all members
benchmark --endpoints=${HOST_1},${HOST_2},${HOST_3} --conns=100 --clients=1000 \
    put --key-size=8 --sequential-keys --total=100000 --val-size=256
```

2.2 APISIX 配置

2.2.1 APISIX 配置文件详细解释

> 参照配置文件apisix.yaml

2.2.2 APISIX 集成 Dashboard 配置详细解释

> 参照配置文件dashboard.yaml

2.2.3 APISIX 集成 prometheus 和 grafanna 配置

- apisix 配置prometheus端点 export 给prometheus,直接配置其插件属性即可
- prometheus 配置 job 相关内容,读取指标
- grafana 接入 prometheus 作为datasource 配置适合apisix 的dashboard(官方已经提供模板)
- dashboard 通过iframe 内嵌,如果系统已经有grafana 请添加grafana_dashboard.json

2.3 灰度插件 traffic-split 使用

官方文档